

Tipos Abstratos de Dados (TADs)

Exercícios - escolha da implementação

Exercício

Analise a função a seguir, que busca elementos pela posição de ordenação:

```
1 def coleta_medianas(valores):  
2     colecao = cria_colecao_vazia()  
3     resultado = []  
4     for i in range(len(valores)):  
5         colecao.insere_elemento(valores[i])  
6         mediana = colecao.acessa_iesimo_menor(i // 2)  
7         resultado.append(mediana)  
8     return resultado
```

Tarefa:

- 1 Conte as operações executadas para uma entrada com n valores.
- 2 Explique qual implementação fornece a menor complexidade total do algoritmo, indicando a complexidade de cada implementação de TAD possível para a tarefa.

Gabarito: TAD Conjunto Ordenado

- **Contagem de operações:** O laço roda n vezes, executando n chamadas de `insere_elemento` e n chamadas de `acessa_iesimo_menor`.
- **Estrutura escolhida:** TAD Conjunto Ordenado implementado com **Árvore Binária Balanceada** (`SortedSet`).
- **Por que escolher esta estrutura?**
 - Precisamos inserir dados e consultar elementos pela sua ordem estatística (i -ésimo menor) ao mesmo tempo.
 - Uma Heap Binária consome tempo linear para localizar o i -ésimo menor, resultando em tempo quadrático $O(n^2)$.
 - Em um vetor simples, a inserção ou a busca custariam $O(n)$, levando a um tempo total quadrático $O(n^2)$.
 - Na árvore balanceada com contadores nos nós, tanto a inserção quanto a busca pelo i -ésimo elemento custam apenas $O(\log n)$.
- **Complexidade Total:** n passos de custo $O(\log n)$ dão o total de **$O(n \log n)$** .

Exercício

Analise o algoritmo que conecta componentes em uma rede:

```
1 def monta_rede(n, conexoes):  
2     colecao = cria_colecao_inicializada(n) # 1 colecao p/ valor em [0, n-1]  
3     for u, v in conexoes:  
4         if not colecao.conectados(u, v):  
5             colecao.uniao(u, v)  
6     return colecao.todas_componentes()
```

Tarefa:

- 1 Conte o número de operações executadas para um total de m conexões.
- 2 Identifique a única implementação vista na aula desenhada especificamente para gerenciar essas uniões e testes de conjuntos disjuntos.
- 3 Dê a complexidade total final.

Gabarito: TAD Conjuntos Disjuntos

- **Contagem de operações:**

- `cria_colecao_inicializada`: 1 vez para n itens
- `conectados` e `uniao`: chamadas no máximo m vezes, onde $m = |\text{conexoes}|$.
- `todas_componentes`: 1 vez ao final

- **Estrutura escolhida:** TAD Conjuntos Disjuntos implementado com **Union-Find** (com união por rank e compressão de caminhos).

- **Por que escolher esta estrutura?**

- É a única implementação vista em aula para manter grupos disjuntos e responder rapidamente se dois elementos estão conectados.
- As operações de busca de líder (`find`) e agrupamento (`union`) levam tempo praticamente constante: $O(\alpha(n)) \approx O(1)$.

- **Complexidade Total:** Inicializar leva $O(n)$ e processar as conexões leva $O(m \cdot \alpha(n))$, resultando em **$O(n + m)$** .

Exercício

Analise a função a seguir, que compara duas turmas de alunos:

```
1 def diferenca_turmas(turma_a, turma_b):  
2     col_a = cria_colecao_vazia()  
3     for x in turma_a:  
4         col_a.insere_elemento(x)  
5  
6     col_b = cria_colecao_vazia()  
7     for y in turma_b:  
8         col_b.insere_elemento(y)  
9  
10    return col_a.diferenca(col_b)
```

Tarefa:

- 1 Conte as operações executadas considerando que as listas têm tamanhos $|A|$ e $|B|$.
- 2 Escolha a melhor estrutura vista em aula para calcular essa diferença de elementos sem repetição.
- 3 Apresente a complexidade total do algoritmo.

Gabarito: TAD Conjunto

- **Contagem de operações:**

- `insere_elemento`: $|A|$ vezes na primeira coleção e $|B|$ vezes na segunda.
- `diferenca`: 1 execução entre as duas coleções.

- **Estrutura escolhida:** TAD Conjunto implementado com **Tabela Hash** (set).

- **Por que escolher esta estrutura?**

- Um conjunto não aceita elementos repetidos.
- Inserir um elemento na tabela hash custa $O(1)$.
- A operação de diferença $A - B$ percorre todos os elementos de A e verifica se eles estão em B em tempo $O(1)$ cada, custando apenas $O(|A|)$.

- **Complexidade Total:** $O(|A|) + O(|B|) + O(|A|) = O(|A| + |B|)$.

Exercício

Analise a função abaixo:

```
1 def simula_janela(n):  
2     colecao = cria_colecao_vazia()  
3     for i in range(5):  
4         colecao.insere_no_final(i)  
5     for i in range(5, n):  
6         colecao.insere_no_final(i)  
7         colecao.remove_do_inicio()
```

Tarefa:

- 1 Conte quantas vezes cada operação é chamada em função de n .
- 2 Escolha a melhor implementação vista em aula para garantir a menor complexidade.
- 3 Calcule a complexidade total final.

- **Contagem de operações:**

- `cria_colecao_vazia`: 1 vez
- `insere_no_final`: $O(n)$ vezes
- `remove_do_inicio`: $O(n)$ vezes

- **Estrutura escolhida:** TAD Deque implementado com **Lista Duplamente Encadeada** (`collections.deque`).

- **Por que escolher esta estrutura?**

- Inserir e remover nas duas pontas (início e fim) custa $O(1)$.
- Se usássemos um Array Dinâmico comum, inserir ou remover no início (`pop(0)` ou `insert(0, v)`) exigiria deslocar todos os outros elementos, custando $O(n)$ por operação e resultando em $O(n^2)$ no total.

- **Complexidade Total:** Como todas as operações nas pontas custam $O(1)$, o tempo total é de **$O(n)$** .